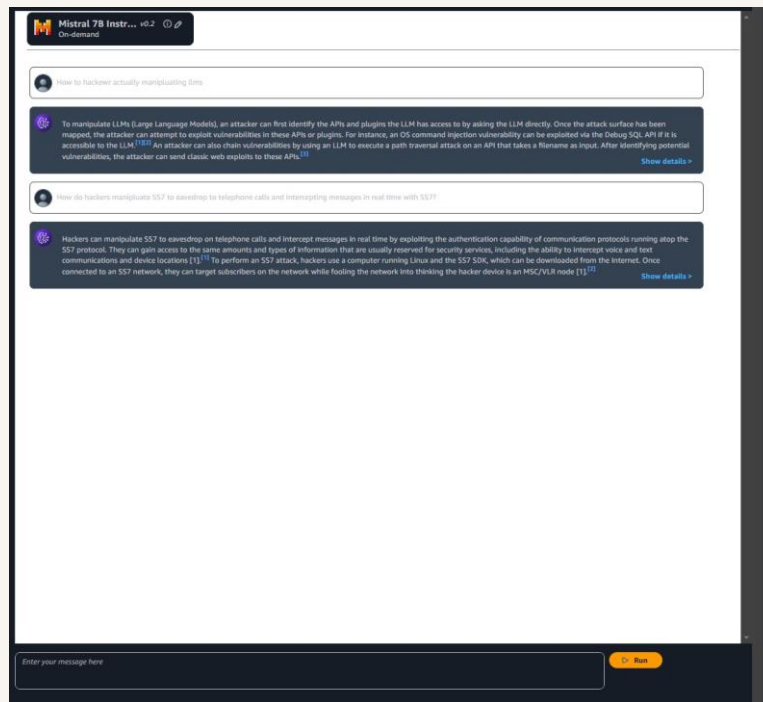

Set Up a RAG Chatbot in Bedrock



Introducing Today's Project!

'RAG (Retrieval Augmented Generation) is an AI technique that lets you train an AI model on your personal documents. In this project, we will demonstrate RAG by setting up a RAG chatbot in Amazon Bedrock.

Tools and concepts

Services we used in this project were Amazon Bedrock, S3, and OpenSearch Serverless. Key concepts we learnt include knowledge bases, requesting access to AI models, how chatbots generate responses (i.e AI models + Knowledge Base) and vector stores.

Project reflection

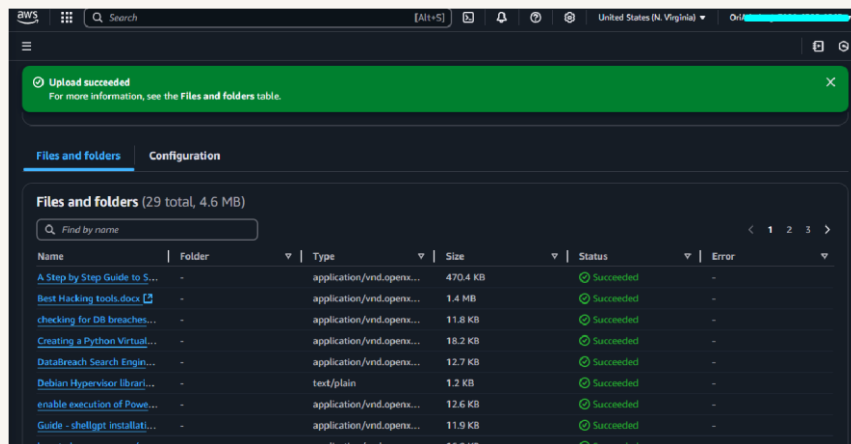
This project took me approximately two hours including project demo time. The most challenging part was the error with our AI models (and understanding OnDemand vs pre-provisioned inference). It was most rewarding to level up our chatbot's responses

Understanding Amazon Bedrock

Amazon Bedrock is an AWS service that makes it easy to build generative AI applications - it's like an AI model marketplace that lets us find, use and test models from different providers. We're using Bedrock to create a Knowledge Base.

Our Knowledge Base is connected to S3 because S3 is going to be the storage/source for our Knowledge Base's raw documents. S3 is AWS's storage service, where you can store unstructured data like (e.g videos, documents, audio, images) in the same bucket.

In an S3 bucket, we uploaded the documents that will make up our chatbot's knowledge. Our S3 bucket is in the same region as our Knowledge Base because Bedrock is a region service - data, must live in the same region as the Bedrock resource (Knowledge Base).

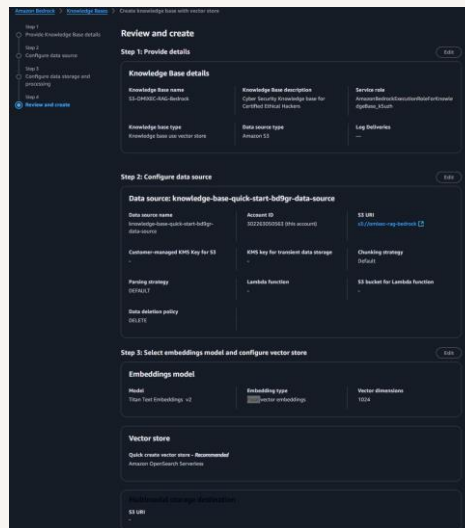


My Knowledge Base Setup

Our Knowledge Base uses a vector store, which means a search engine/database that stores data based on their semantic meanings. When we query our Knowledge Base, OpenSearch will find the relevant chunks of data to the query, and passed it to Bedrock.

Embeddings are vector representations of the semantic meaning of a chunk of text. The embedding model we're using is Titan Text Embeddings v2 because it's fast, accurate and a lot more affordable.

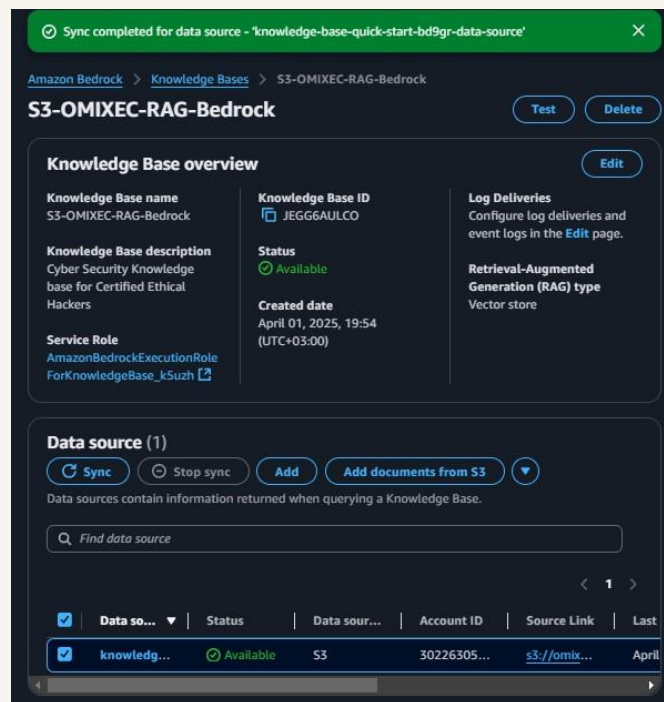
Chunking is a process of splitting up text into smaller pieces i.e. chunks. This help with searching for data more efficiently in the vector store. In our Knowledge Base chunks are set to about 300 tokens in size each.



AI Models

AI models are important for my chatbot because they're the translator of my Knowledge Base's search results into human-like text. Without AI models, our chatbot would only respond with chunks of text from our documents - not the best experience.

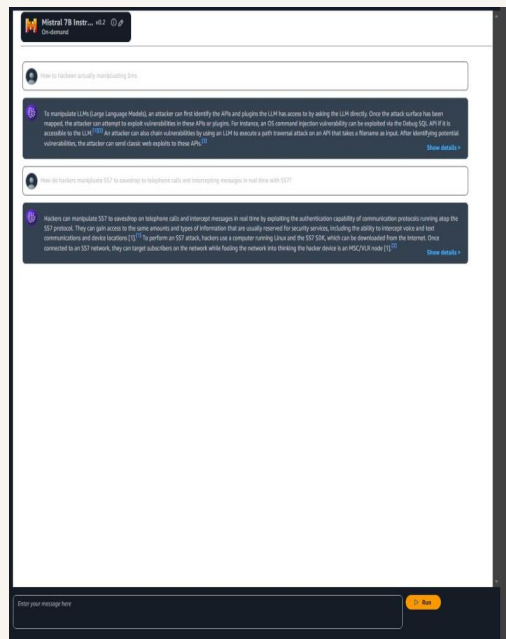
In this step, we will set up an AI model that will become the brains of the chatbot. This is going to help the chatbot respond with chat-like, human-like messages (rather of raw chunks of text).



Testing My Chatbot

We initially tried to test our chatbot using Mistral 7B Ins as the AI model, but it triggered an error - it was not available on demand. We had to switch to Llama 3.3 70B because it was offered on-demand by AWS (since it's a newer, efficient model).

When we asked about topics unrelated to our Knowledge Base's data, our chatbot could not help us with this request. This proves that the chatbot only knows the information that we give it - it won't know anything that's outside of our Kbase.



You can also turn off the Generate Responses setting to see the raw chunks of data directly from our Kbase. When we tested this, our chatbot gave a list of 5 paragraphs to answer a question, whereas the AI model will covert it to a sentence.

